

Data Structure(CSL202)

By-
Dr. Sk Subidh Ali (subidh@iitbhilai.ac.in)



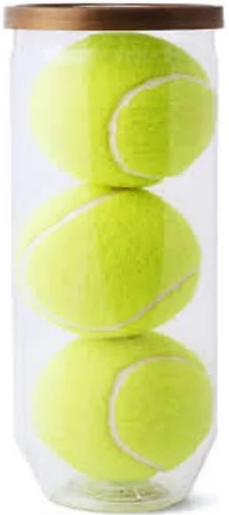
Stack

Stack

- A Stack is a linear data structure that follows the Last In, First Out (LIFO) principle.



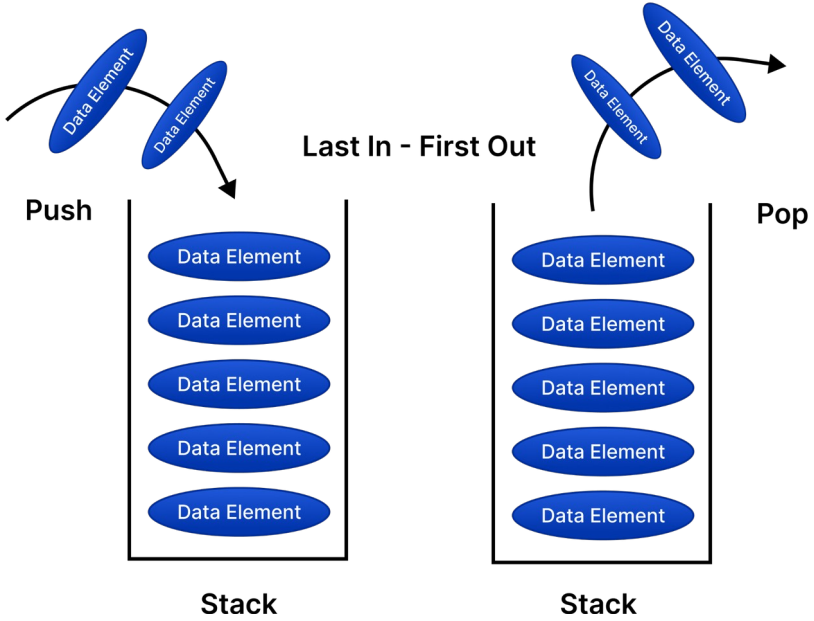




Stack Operations

- **Push:** Adds an element to the top of the stack.
- **Pop:** Removes the top element from the stack.

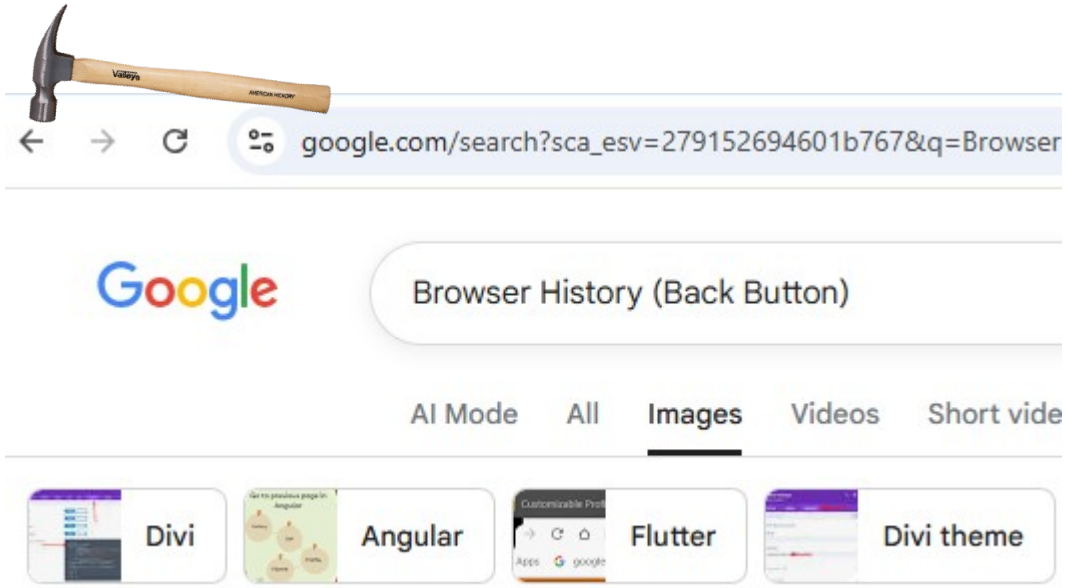
Top: Pointer where the next operation will occur.



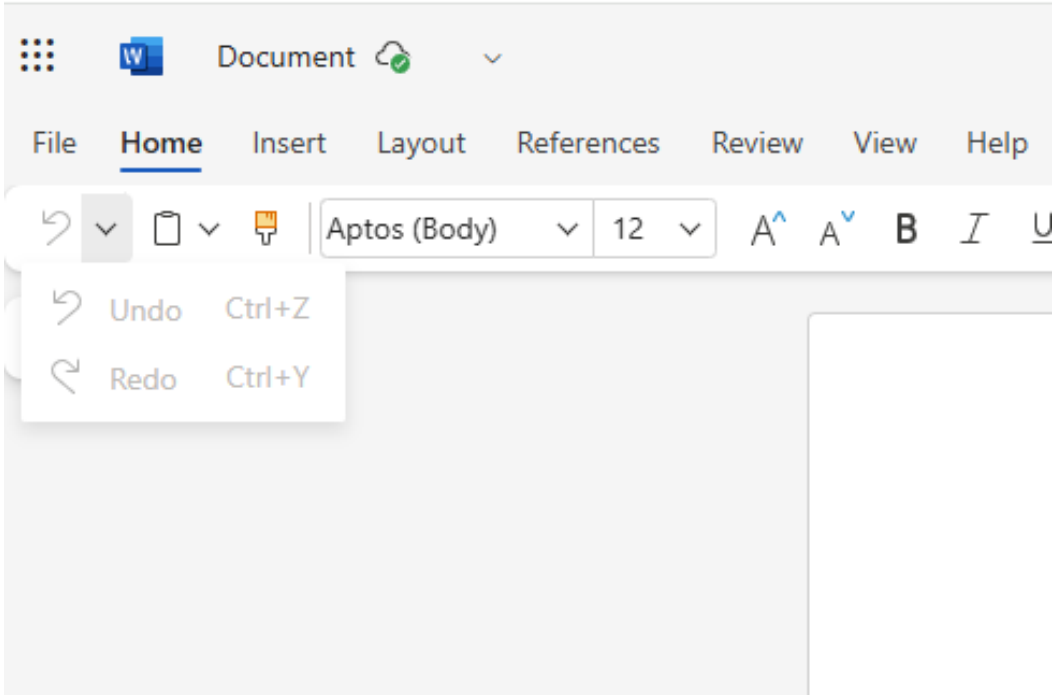


Applications of Stacks

Browser History (Back Button)



Undo Operation in Text Editors



HTML/XML Tag Matching

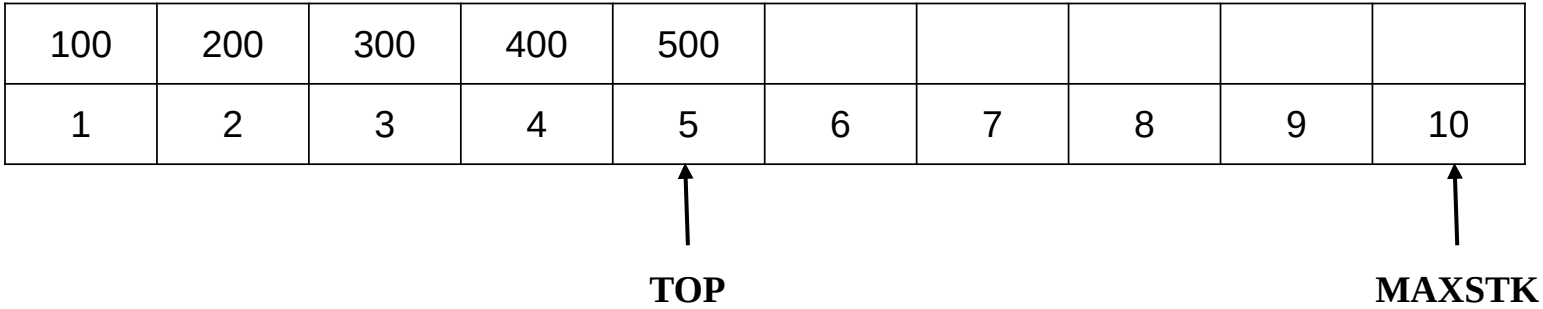
- Browsers use a stack to verify if opening and closing tags are correctly nested.

```
<!DOCTYPE html>
<html>
    <head>
        <title>Simple HTML Program</title>
    </head>
    <body>
        <header>
            <h1>Hello, World!</h1>
        </header>
        <footer>
            <p>&copy; 2024 Simple HTML Program</p>
        </footer>
    </body>
</html>
```



Representation of Stacks

- Stack is stored using a linear array STACK[ ].
- TOP points to the top element's index.
- MAXSTK stores the maximum size.
- TOP = 0 or NULL means stack is empty..



PUSH Operation

- Check for Overflow: If TOP == MAXSTK, print "OVERFLOW".
- Increment TOP by 1.
- Place ITEM at STACK[TOP].
- **Time Complexity:** O(1) in all cases (constant time insertion).

100	200	300	400	500					
1	2	3	4	5	6	7	8	9	10

TOP **MAXSTK**

PUSH 150

TOP + 1

100	200	300	400	500	150				
1	2	3	4	5	6	7	8	9	10

POP Operation (Delete)

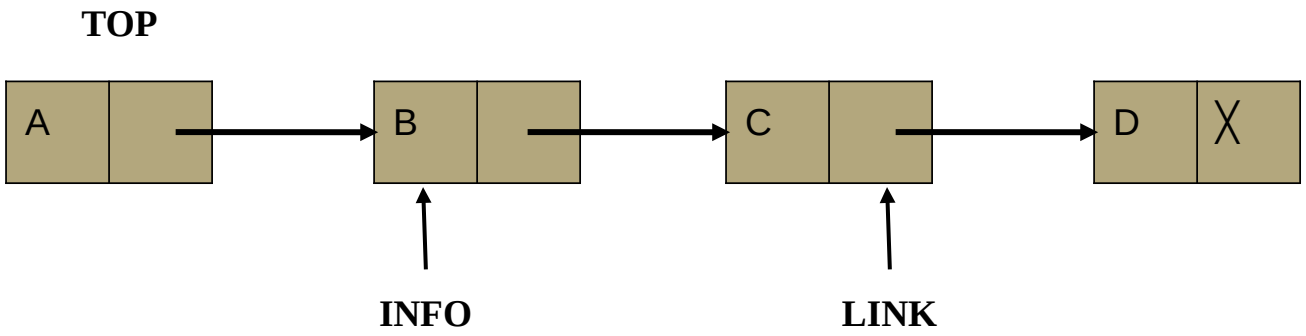
- Check for Underflow: If TOP == 0, print "UNDERFLOW".
- Assign STACK[TOP] to ITEM.
- Decrement TOP by 1.
- **Time Complexity:** O(1) in all cases (constant time removal).

100	200	300	400	500	150				
1	2	3	4	5	6	7	8	9	10

PUSH **TOP - 1** **TOP** **MAXSTK**

100	200	300	400	500					
1	2	3	4	5	6	7	8	9	10

- Using a singly linked
- Each node has:
  - **INFO field:** stores the data (stack item).
  - **LINK field:** pointer to the next node.
- TOP pointer points to the top of the stack (like START of linked list).
- No size limit (unlike array stacks).

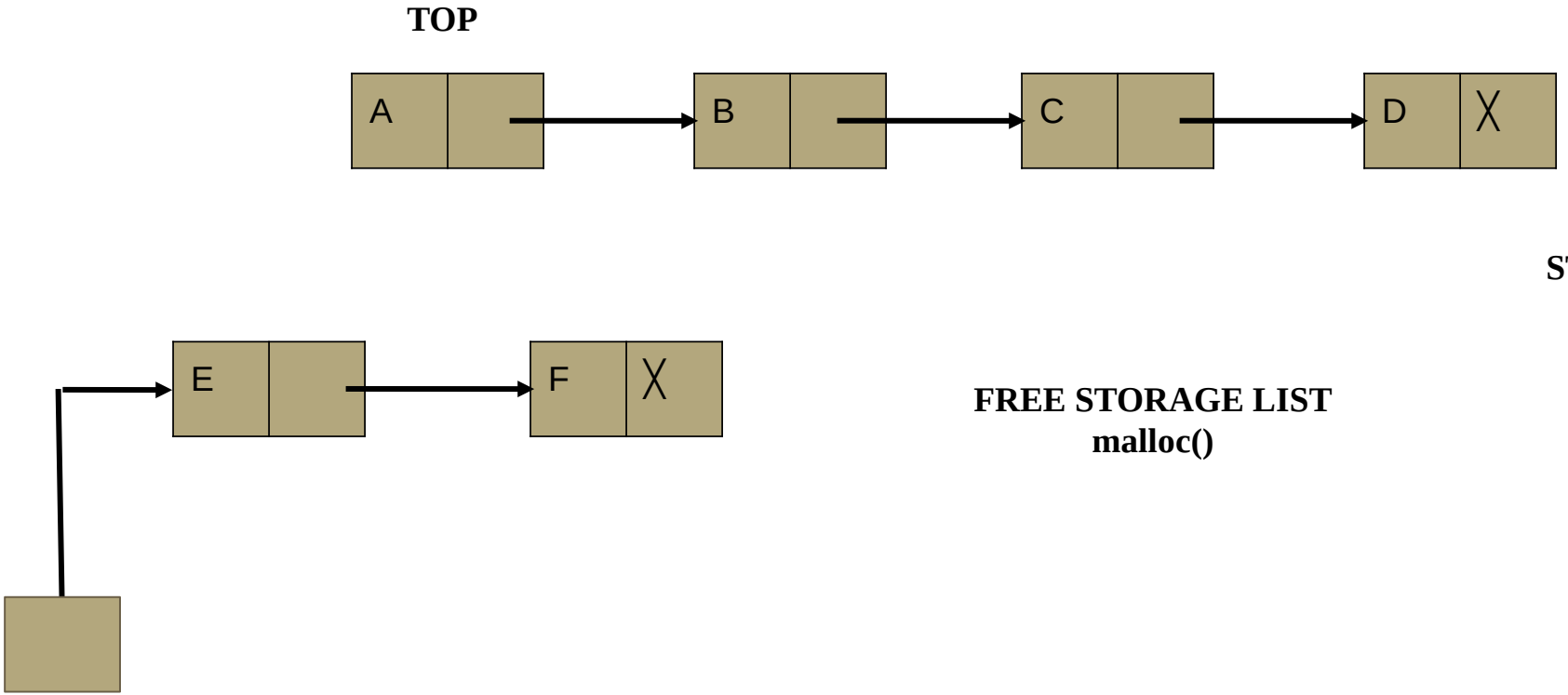


PUSH_LINKSTACK

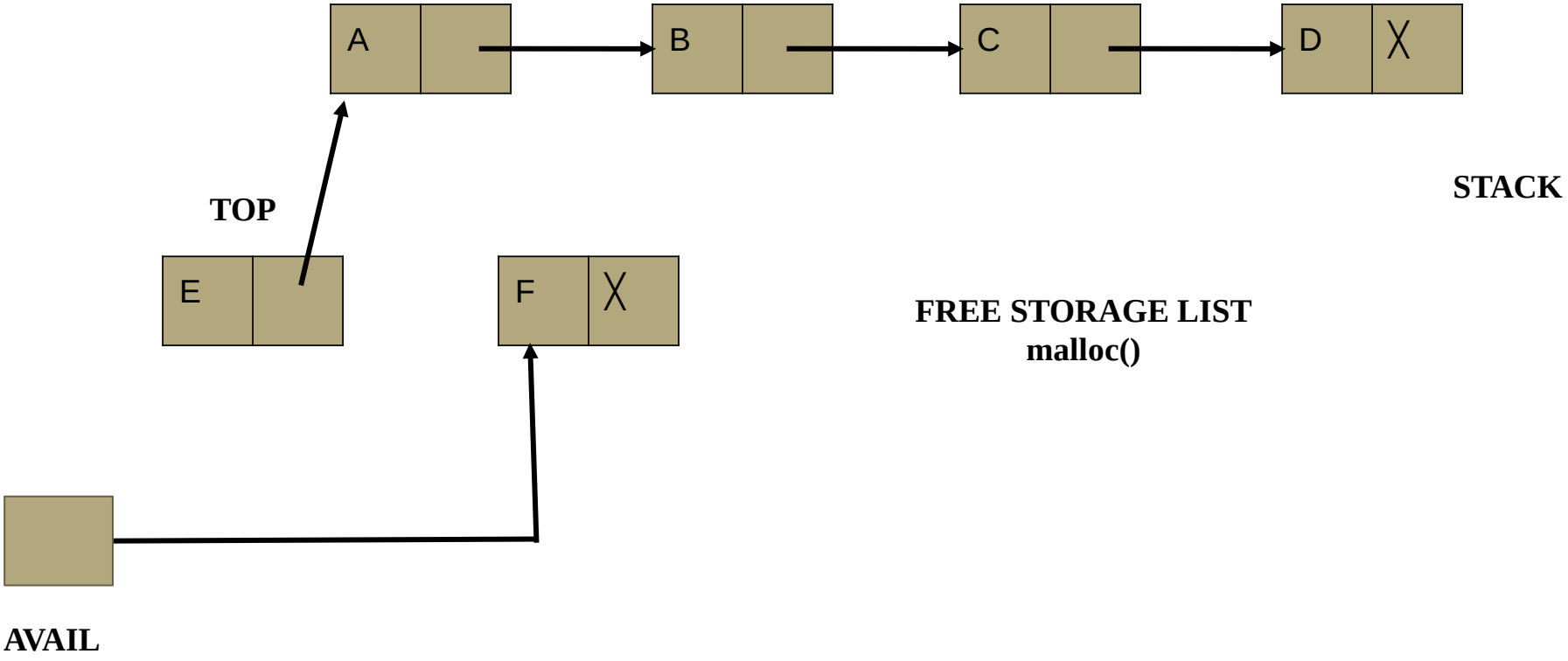
- Check Overflow: If AVAIL = NULL, print "OVERFLOW" and exit.
- Allocate Node:
  - NEW = AVAIL.
  - AVAIL = LINK[AVAIL].
- Store Item in INFO[NEW].
- Point New Node to Previous Top using LINK[NEW] = TOP.
- Update TOP to NEW.
- Exit.

Time Complexity: O(1) Adding a node at the head (constant time).

Linked Representation of Stacks (PUSH Operation)



Linked Representation of Stacks (PUSH Operation)

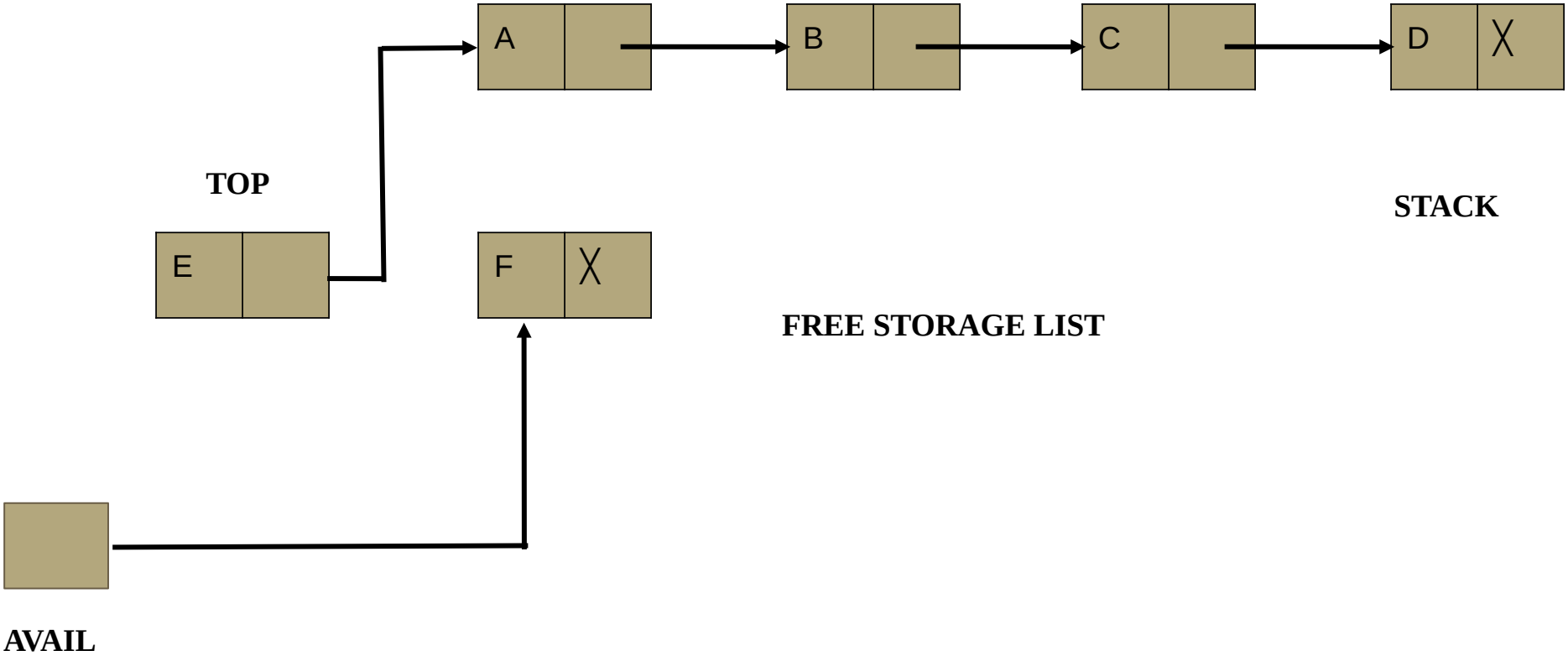


POP_LINKSTACK:

- Check Underflow: If TOP = NULL, print "UNDERFLOW" and exit.
- Copy Top Item: ITEM = INFO[TOP].
- Update TOP:
  - Save old TOP in TEMP.
  - Move TOP to LINK[TOP].
- Return Node to Free List:
  - LINK[TEMP] = AVAIL.
  - AVAIL = TEMP.
- Exit.

Time Complexity: O(1) Removing the top node (constant time).

Linked Representation of Stacks (POP Operation)



Linked Representation of Stacks (POP Operation)

